SkeleTrace 0.13.0

A Technical Assessment of the Standalone OSINT Engine

Engineering white paper | Assessment 1.0 | 5 October 2026

Claudia G. Petersen

Analysis, test execution and document production assisted by OpenAI ChatGPT. This is a maintainer-directed assessment of a fixed source release, rather than independent certification or peer review.

Abstract

SkeleTrace 0.13.0 is a Rust engine and command-line interface for source-neutral OSINT telemetry. It combines identity-bearing graph entities, typed metrics, current-state queries, selective history, local persistence and requested view materialization. The standalone release retains these mechanisms while removing embedded dependencies on Maxwell's Demon Detector, OSINTropy and their command/API surfaces. CrowdWatch integration was already absent from the supplied code.

This assessment combines source inspection, a fresh locked build, 114 passing tests, one passing documentation example, an offline workflow and controlled measurements across 90 independent processes. On the specified Linux host, 5,000 synthetic entities and 50,000 normalized observations were ingested in a median 28.90 ms with RAM history, or 655.51 ms with SQLite history. Full latest-value queries took 2.91 ms with RAM history; the SQLite case's median process high-water memory was 28.39 MiB.

The evidence supports a compact, useful engine for controlled and embedded workflows. It also identifies specific limits in history checkpoint semantics, memory-budget enforcement, source-failure isolation and dependency maintenance. Results apply to the defined fixtures and environment; they establish no universal cost saving, hard RAM limit or unrestricted production guarantee.

At a glance

- **Core preserved:** graph, metrics, adapters, queries, watchlists, SQLite, snapshots and structural topology analysis.
- **Build simplified:** 8 direct dependencies and 155 locked packages, down from 14 and 217 in the supplied integrated input.
- **Evidence supplied:** source identity, check logs, a complete measurement harness, raw results and observed limitation diagnostics.

1. What the earlier paper actually assesses

The supplied 14-page paper, *Sparse Spatiotemporal OSINT Flow-Graph Engine: Comprehensive Technical Analysis & Development Report*, is dated 24 March 2026. It contains no package version, commit, source digest, benchmark data or scoring rubric. Its references to stage 10 benchmarking, stage 11 queries and Lookyloo are consistent with the historical engine lineage. They do not identify one exact build. The date alone cannot pin a release. [1]

The present assessment targets the already-delivered **0.13.0 standalone archive**, whose manifest and complete file contents were verified. The supplied predecessor was labelled 0.12.0; an earlier cleanup check reproduced a missing `osintropy` module error in that input. That is evidence about the supplied integrated tree, not evidence that the March paper's unnamed build failed. No historical speed-up is claimed. [2, 3]

Earlier statement	What 0.13.0 actually provides	Assessment
Sparse entities with optional geography	Typed nodes, edges, boundaries, metrics and optional geographic positions	Implemented
Three-pane graph, flow and globe display	Topology/GeoJSON payloads, data cards and analysis APIs; no supplied graphical front end	Presentation remains a host responsibility
Geo, topology and null coordinate spaces	Optional geographic positions and graph relationships; Cartesian helpers exist, but no node XY layout model	Architectural idea, not three complete renderers
Hot/warm/cold storage	Hot cache, retained in-memory samples, optional SQLite/NDJSON and explicit snapshots	Implemented with resource and durability limits
Compressed Parquet/GeoJSON exports	Native JSON, CSV and GeoJSON	Parquet and built-in export compression are absent
Automatic retry, rate limit and circuit breaking	A reusable TransportClient exists; built-in HTTP/Lookyloo adapters use the simpler one-shot helper	Capability exists, wiring differs
60-80% storage/cost savings	One matched-input experiment is reported in Section 7	No general cost conclusion
89% overall readiness; production deployment recommended	No documented basis for the old percentages	Replaced by evidence and scoped decisions

The earlier document is useful as a record of design intent. Its competitive, financial and readiness claims are not a reliable benchmark against which to award the current release a higher percentage.

2. Architecture of the standalone engine

SkeleTrace's central design choice remains sound: **store observations and relationships, then materialize the requested view**. Geographic context is optional. An abstract identity or relationship does not require a fabricated location to participate in the graph. [4]

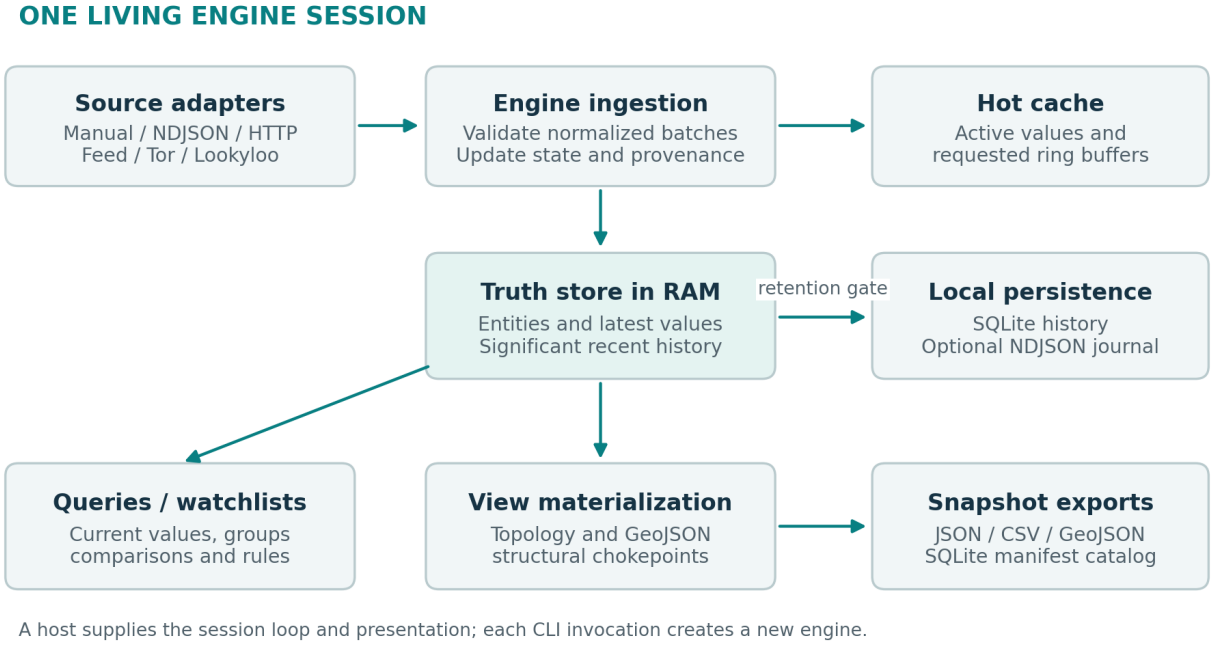


Figure 1. The ingestion and output paths in 0.13.0. Persistence is optional; this diagram describes components rather than background services. Raw journal inputs can be written independently of sample retention.

Nodes, edges and boundaries have typed identifiers and metadata. Metric definitions specify value type, cadence, interpolation policy and retention. Samples carry observed/ingested timestamps, a source identifier and quality. Current values are indexed by entity/metric pair; recent retained history is stored separately. The public Flow contract describes a path and time window, but the distribution supplies no complete flow dashboard or autonomous flow-analysis service. [4]

The hot cache holds active values and bounded ring buffers requested for detailed views. SQLite provides indexed retained history; an optional NDJSON journal can record raw inputs and retained samples. Snapshot exports make selected investigation data portable. This is a set of cooperating local mechanisms, not a guarantee that every record moves through three enforced storage tiers. [4, 5]

The session boundary is essential. Every CLI invocation instantiates a fresh engine. Latest values, cache entries, schedules and adapter read offsets belong to that process. Opening an existing SQLite history file does not reconstruct all live state. A continuous ingest/query/view workflow must keep one SkeletraceEngine or OperatorApi alive. The supplied offline example demonstrates this complete session. [3, 4]

3. Implemented capabilities and release improvements

Area	Evidence in 0.13.0	Practical value
Acquisition boundaries	Manual push, NDJSON, mapped HTTP JSON, RSS feed, Tor-route and Lookyloo adapters	Sources can feed the engine without linking whole applications
Operator queries	Entity/source/status/tag filters, sorting, grouping, saved query shapes and history comparisons	Reuse structured investigations and produce current-state reports
Watchlists	Numeric, code, flag and presence predicates	Evaluate explicit rules against current values
Requested views	Topology closure and viewport-filtered GeoJSON feature payloads	A host can build a focused presentation from a selected slice
Structural analysis	Iterative articulation-point and bridge analysis	Identify connectivity weak points in the observed graph
Local exports	JSON, CSV, GeoJSON and optional SQLite manifest catalog	Preserve selected findings in familiar formats
Operational evidence	Health reports, bounded events, audit/provenance and replay workloads	Inspect behaviour without contacting live sources

Lookyloo support includes capture summaries and topology materialization, using stable source-derived identities. These adapters contact an operator-configured Lookyloo service; they do not embed or install that service. Provider-specific OSINTropy search connectors and the detector's packet/UI functions are outside this release. [4, 6]

The separation also repairs concrete hot paths. RingBuffer::latest now directly indexes its most recent slot without allocating. Topology closure borrows stored edges and uses hash sets for membership checks. Latest-value queries borrow the index and clone only emitted values. Snapshot stems are confined to portable filenames, and CLI error statuses and broken-pipe behaviour have explicit tests. [3, 4]

Dependency declarations fell **42.9%** (14 to 8); locked package entries fell **28.6%** (217 to 155). These are graph counts, including target-specific lockfile entries, rather than measured binary-size or vulnerability reductions. The optimized Linux CLI measured **5,766,144 bytes**. No equivalent working historical binary was available for comparison. [2, 3]

4. Validation and evidence quality

All checks below were run against the fixed standalone source with Rust 1.87.0 on Linux x86_64. The source manifest matched before and after execution, and every regular file in the delivered archive matched the inspected working tree. No application code was changed for this assessment. [2, 7]

Check	Result	What it supports
Locked tests, all targets	114 passed; no failures or ignored tests	Existing and separation-specific behavioural contracts
Documentation tests	1 passed	The executable documentation example
Formatting and clippy	Passed; warnings denied	Consistent formatting and static lint discipline
Optimized build, all targets	Passed	Release compilation with LTO and one codegen unit
Offline workflow	Passed	Ingest, query, views, watchlist, snapshot and SQLite catalog in one session
CLI smoke exercise	All 11 engine commands passed	Command/request compatibility with generated fixtures
Standalone isolation audit	32 reachable source files; no reported orphans/integration dependencies	The intended application separation
Large-chain topology contract	200,000 nodes passed	Iterative traversal avoids recursive stack exhaustion for that fixture

The offline workflow ingested three synthetic samples, returned three query rows, found one chokepoint and two bridges, and wrote a geographic snapshot/catalog. The measurement harness separately checks expected graph, latest-value and history counts on every workload. Four diagnostic assertions reproduce observed limits rather than counting them as product correctness tests. [7, 8]

These results are **not a line-coverage percentage**, proof of error-free software, external-service acceptance test or native Windows validation. Several acquisition tests use local fixtures or local HTTP endpoints. The current assessment makes no claims about remote provider availability, live Tor anonymity, paid API acceptance or identity-attribution accuracy.

The March paper's five confidence percentages and readiness scores had no documented calibration. This assessment uses named checks, raw output and explicit interpretation so a reader can evaluate the evidence directly.

5. Measurement method

The controlled experiment used **50, 500 and 5,000 entities**, each arranged in an undirected path with N-1 edges. Each entity had a geographic position and one numeric metric. One manual source supplied ten preconstructed batches, one observation per entity per batch, with one-second timestamp spacing. No records concerned real people, organizations or infrastructure. [8]

Variable	Definition
Storage	RAM retained history; or RAM plus a new SQLite history file
Changing fixture	Values rise 10 units each batch; every sample stored
Constant control	Identical constant observations with every sample stored
Constant gated fixture	The same observations; change-only enabled with absolute threshold 5 and silent-gap parameter 60 seconds
Repetition	5 independent processes per size/storage/pattern cell: 90 processes total
View/query timing	20 calls per operation in each process; report median of the five process medians
Ingestion timing	One timed ten-batch run per process; report median of five runs
Memory	Linux VmHWM read after measured operations; includes engine, prebuilt fixtures, outputs and allocator overhead

Each process constructed a fresh engine, and SQLite cases used distinct new databases. Operating-system caches were not flushed. Ingestion timing includes batch validation, store/cache updates, provenance, source bookkeeping and pruning, but excludes fixture construction and external parsing/acquisition. View timers exclude JSON serialization, snapshot writes and front-end rendering. Geographic views request N point nodes; edge IDs are not included in that view. [8]

The active-cache count was raised to max(1,024,N) to retain the benchmark's entities; other engine defaults remained. Metric hot/warm retention was 600/3,600 seconds. No history ring buffers were promoted. These settings must accompany any quotation of the results.

The host was a virtualized AMD EPYC 9V74 system, Linux 6.18.44 x86_64, with 9 visible logical CPUs and a cgroup quota equivalent to 8 CPU cores. Builds used Rust/Cargo 1.87.0, the locked application dependency graph, opt-level 3, LTO and one codegen unit. The synchronous measurement path did not create a parallel ingestion worker pool. [8]

6. Controlled performance results

The changing fixture stores all ten samples per entity. Values below are milliseconds, not CPU utilization percentages. They characterize normalized engine work on this host. [8]

Entities	Samples	RAM ingest ms	SQLite ingest ms	RAM latest query ms
50	500	0.243	7.333	0.015
500	5,000	2.072	63.704	0.205
5,000	50,000	28.902	655.511	2.911

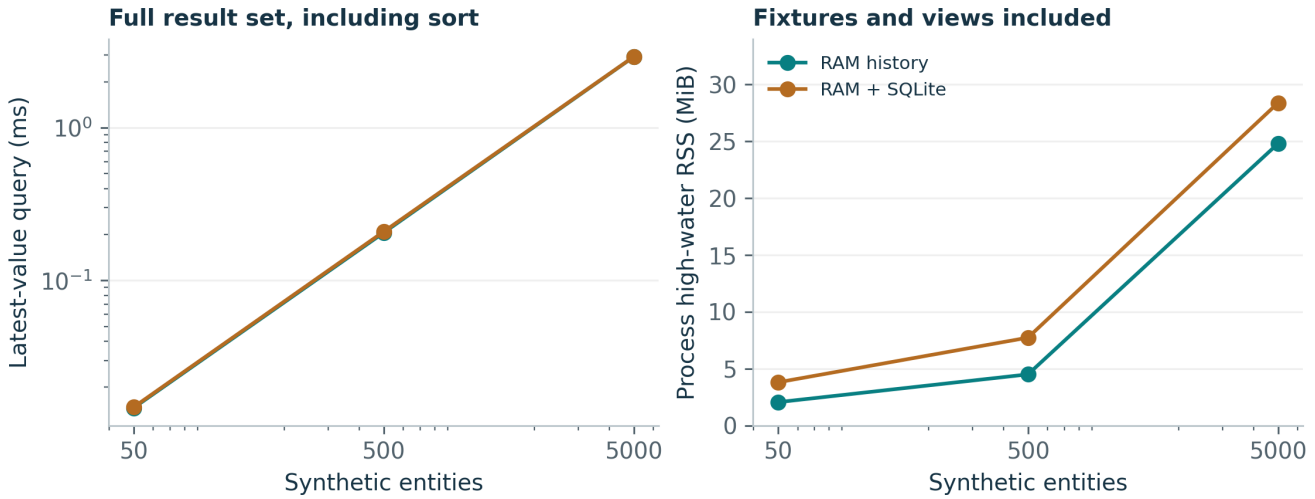


Figure 2. Median full-result query latency and process high-water memory, changing fixture. The query panel uses logarithmic axes; memory is linear. Each point summarizes five independent processes.

Entities	Topology ms	Chokepoint/bridge ms	Geographic points ms
50	0.025	0.011	0.017
500	0.275	0.097	0.196
5,000	3.283	1.067	2.179

At 5,000 entities, the full RAM-backed latest query's five process medians ranged from 2.50 to 3.00 ms; SQLite-backed ingestion ranged from 629.74 to 719.18 ms. Sparse geographic output is an in-memory feature payload, not a complete map-rendering latency. The supplied CSV includes min/max values for every cell.

7. Memory and selective-history behaviour

Entities	RAM peak MiB	RAM + SQLite peak MiB	SQLite five-run range MiB
50	2.05	3.80	3.74-3.85
500	4.50	7.73	7.64-7.74
5,000	24.81	28.39	28.35-28.42

The measured footprints are numerically below the March paper's stated targets of 50 MB, 200 MB and 1 GB for these entity counts. Those older figures were targets, not measurements. The present fixtures use one numeric metric and short labels; other cardinalities, payloads, history lengths, concurrency and views can produce different memory use. Cache estimates are not measurements of total process RSS. [1, 8]

The matched constant-data control isolates the gate's effect. Both 5,000-entity SQLite cases ingest the same **50,000 observations**; one stores every sample, while the other enables change-only retention.

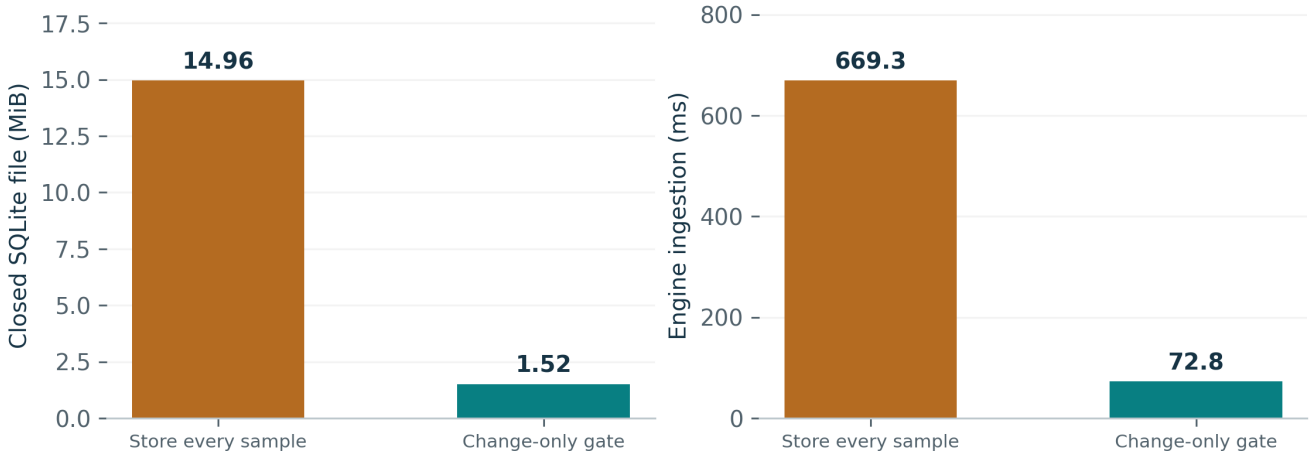


Figure 3. Identical input, different retention setting. Every-sample storage retained 50,000 rows; the gate retained 5,000. The closed database fell from 14.96 to 1.52 MiB (89.8% smaller).

Ingestion took 669.32 ms without the gate and 72.76 ms with it. Row count fell **90%**, while current values still covered all 5,000 entities. This supports selective persistence for redundant observations in this short fixture. It is not a general 90% saving or an infrastructure-cost estimate. Closed-file size excludes the live WAL; cumulative physical write traffic was not measured. [8]

The same reduction must not be confused with a reliable periodic-history guarantee. Section 8 documents a checkpoint limitation discovered in a longer diagnostic.

8. Observed runtime limits

History checkpoints compare consecutive observations. RetentionPolicy::should_store receives the previous latest value, and every valid observation refreshes that latest entry. Therefore the time-gap test measures time since the previous observation, rather than time since the last retained sample. [4, 8]

With a 60-second gap setting, 121 identical observations arriving one second apart retained only the first sample. A second diagnostic used values 1,000 through 1,120 with a per-step delta of 1 and an absolute threshold of 5: again only the first sample was retained, while the current value correctly became 1,120. Frequent small updates can therefore leave a sparse or misleading historical record. Hosts needing every checkpoint or cumulative drift evidence should use store-every-sample retention until those semantics are deliberately corrected.

Memory limits are partly soft. The byte/high-resolution/ring-count budget fields are not comprehensively enforced. Selected or visible cache entries are protected from count-based eviction: the diagnostic configured an active limit of one and obtained two visible entries. Entity registries, latest values and retained samples also live outside that cache budget. There is no hard total-process RAM guarantee. [3, 4, 8]

Source failure can end a tick early. The synchronous tick loop returns on the first failing due source; later due sources in that tick are not all guaranteed to run. Two due failing adapters resulted in one adapter call. Backoff and failure evidence exist, but this is not complete fault isolation. A host can poll and handle sources individually when continuation is required. [4, 8]

The richer transport client is not automatic. Built-in HTTP JSON, feed and Lookyloo adapters call http_get_text, a helper explicitly documented as having no rate limiter, retry or circuit breaker. Those mechanisms are available in TransportClient for a host/custom adapter; engine-level scheduling backoff does not add the entire client policy to each request. Tor routing likewise requires a suitable running proxy. [6]

Analysis has a defined mathematical scope. The chokepoint traversal treats the materialized graph as undirected connectivity. It does not infer traffic capacity, causal importance, hostile intent or future movement. Its DFS core is O(V+E); output ranking and sorting add costs. Latest queries scan the index and sort returned rows. No constant-efficiency or unlimited-scaling claim follows from the measurements. [4]

9. Persistence, security and release boundaries

Warm history is configured with SQLite WAL and synchronous=NORMAL. Inserts use bound SQL parameters and an indexed entity/metric/time lookup. Retention pruning and optimize/vacuum controls exist. Batch ingestion iterates individual inserts; it does not promise an atomic transaction across the whole batch and all RAM/journal/SQLite state. [5]

SQLite documents that NORMAL in WAL mode can lose recently committed transactions after power loss or an operating-system crash. Accordingly, this release's configuration must not be advertised as an unconditional durability guarantee. Vacuuming reclaims free space; the source provides no database/export compression subsystem. [5, 9]

A dependency update is required before concurrent WAL deployment. The pinned libsqlite3-sys 0.28.0 bundle contains SQLite **3.45.0**, confirmed from its build-source header. SQLite's official documentation identifies a rare WAL-reset race affecting older releases, fixed in 3.51.3 and certain backports. It requires multiple connections to the same database with overlapping writes/checkpoints. That condition was not exercised by the single-connection measurement cells. A compatible fixed dependency should be selected and the assessment rerun before recommending concurrent writers/checkpointers. [2, 9]

The first-party src tree contains no unsafe keyword. That is a useful inspection result, not an end-to-end proof of memory safety: dependencies include native SQLite and their own low-level implementations. Rust's safe/unsafe distinction also does not establish application correctness, bounded resource use or source-data validity. This assessment is not a comprehensive vulnerability audit. [4, 10]

Source policies are explicit capability-based checks at registration, with permissive defaults. The engine cannot verify that supplied capability labels describe a provider's terms or data correctly. Authentication can contain literal credentials in local profiles. Protect those files and export destinations; configure policy before registering sources. Source adapters make outbound requests to operator-configured endpoints. [3, 6]

The supplied LICENSE text is GPL version 3, while earlier metadata claimed MIT. 0.13.0 preserves that text and references it using license-file. The owner's intended licensing designation should be settled before public licensing statements. Native Windows/MSVC execution and remote-provider acceptance remain unverified here. [2, 3]

10. Assessment and release positioning

Dimension from the March paper	Current evidence-based judgement
Architectural foundation	Coherent source-neutral model and reusable engine boundaries; current session semantics are explicit
Implementation quality	Strong automated-contract evidence for the tested scope; additional diagnostics identify specific unresolved behaviour
Performance optimization	Small footprint and millisecond query/view work on the defined fixtures; no historical or competitor speed comparison
Production hardening	Partial: validation, events, policy, provenance and backoff exist; checkpoint/failure/transport/dependency work remains
Operator experience	Usable library, documented CLI and complete offline example; no bundled desktop dashboard or daemon
Overall readiness	Supported for scoped evaluation and controlled embedding; unattended/concurrent deployment needs targeted hardening

SkeleTrace 0.13.0 has a defensible story: **a compact Rust engine that turns normalized observations into queryable relationships and selective investigation outputs, with evidence readers can reproduce**. Separating the former suite applications reduces coupling and makes each product's acquisition and presentation boundary easier to understand. The release retains useful core work instead of carrying whole applications inside one executable.

The strongest publication claims are the named implemented features, passing checks, dependency graph reduction and precisely scoped measurements. Broad labels such as enterprise-grade, revolutionary, constant efficiency, automated anonymity, regulatory compliance and universal cost savings have not been established by this assessment.

The next engineering work should address the demonstrated checkpoint semantics, connect or explicitly select production transport policy, preserve progress across failed sources, and update the affected SQLite bundle. Native Windows checks and a long-running workload with realistic source diversity should follow. Enforced total-memory accounting, recovery behaviour and durable host orchestration need their own contracts before stronger operating guarantees.

Release description supported by this assessment

SkeleTrace is a standalone Rust engine for OSINT telemetry and graph analysis. It provides typed observations, configurable retention, local SQLite history, structured queries and watchlists, topology/GeoJSON materialization and portable snapshots. Version 0.13.0 removes embedded companion applications while preserving the core engine. A complete offline example and reproducible assessment accompany the source release. Controlled Linux tests show millisecond current-state queries at 5,000 synthetic entities; deployment behaviour and limits are documented openly.

Appendix A. Reproducing the assessment

The complete assessment archive contains two sibling directories: skeletrace-standalone (the unchanged source) and skeletrace-assessment-0.13.0 (this paper, evidence and external measurement harness). The application requires Rust 1.87.0 and a native C compiler; the runner requires Python 3.11 or later. SQLite is bundled. No credentials or external OSINT services are needed for the fixtures.

Windows PowerShell

Install Rust through rustup using the MSVC host and Visual Studio Build Tools with Desktop development with C++ and a Windows SDK. From the directory containing the two extracted folders, run:

```
cd .\skeletrace-assessment-0.13.0
py -3 .\run_assessment.py --repeat 5
```

Linux

Install Rust through rustup and a C compiler/linker. From the directory containing the two extracted folders, run:

```
cd skeletrace-assessment-0.13.0
python3 run_assessment.py --repeat 5
```

The runner executes locked checks, the offline example and CLI smoke script, then builds the separate harness and measures all 18 cells in five processes each. It writes logs, raw JSON, CSV summaries, source hashes and environment metadata into a fresh rerun directory. Existing output directories are refused. Rust versions/dependencies are pinned in the supplied source and harness. On non-Linux hosts, timings and count assertions still run, but the Linux RSS field is null.

The supplied evidence separates **validation/** from **measurements/**. Validation was completed first; the final matched-input experiment was then run with checks skipped to avoid repeating the unchanged application's suite. Both source manifests are byte-identical. A normal full rerun executes all checks and measurements in one run.

The in-application workflow can also be run on Windows with:

```
cd ..\skeletrace-standalone
cargo run --locked --release --example standalone_demo -- .\output\demo
```

That example keeps one engine alive for ingest, query, views and export. Separate CLI polling and querying commands should not be treated as one persistent daemon session. Generated profiles contain absolute local paths and should be regenerated if moved.

Appendix B. Identity, evidence and sources

Fixed identities

Assessed source archive SHA-256

`32c45fc848845df7068b991e75b2d8453c2ab1b87a7cb93a0092a754e2f5fcb9`

Source-manifest file SHA-256

`c3ac917dc89e842393ecb058b126c65d3ead8154b3c11afcf12dabe92be1d092`

Supplied March paper SHA-256

`fc5f4932a27fb25f0d06935d1689ca1d173edbb02f229514a3d1dd69996d78c0`

The source archive contains 121 regular files and is 2,431,542 bytes. The source-manifest JSON records individual Cargo/toolchain and Rust source/test/example digests. It attests to this application release, not to a later integration kit or unexamined development version. [2]

Evidence and reference index

1. Supplied historical paper: docs/claude_s_paper_on_skeletrace.pdf, dated 24 March 2026, 14 pages. Relevant passages: architecture pp. 4-7; performance pp. 8-9; deployment claims pp. 10-14. Exact release not identified.
2. evidence/release-identity.json and both source-manifest.json files; application Cargo.toml, Cargo.lock and LICENSE. The bundled SQLite version comes from libsqlite3-sys 0.28.0/sqlite3/sqlite3.h in the resolved build source.
3. Application README.md, CHANGELOG.md, SECURITY.md and docs/CLEANUP_REPORT.md. Historical input failure excerpt: evidence/historical-input-build-error.txt.
4. Application src/entity.rs, metric.rs, store.rs, cache.rs, engine.rs, materialize.rs, query.rs, operator.rs, view.rs, spatial.rs and topology_analysis.rs. Functions of particular interest: ingest_sample, should_store, query_latest, promote_for_view, tick and analyze_topology.
5. Application src/warm_store.rs, export.rs, snapshot.rs and profile.rs: insert_sample, prune_for_metrics, init_schema and snapshot stem validation.
6. Application src/adapter.rs, lookyloo.rs, governance.rs and transport/: http_get_text, TransportClient, register_source_with_capability and SourcePolicy defaults.
7. evidence/validation/checks.json, logs/, offline-demo-report.json and harness-clippy.log; application tests/ and examples/standalone_demo.rs.
8. evidence/measurements/: summary.csv, summary.json, measurements.json, raw/, diagnostics.json and environment.json. Full executable method: benchmark/src/main.rs and run_assessment.py.
9. SQLite official documentation: PRAGMA synchronous, https://www.sqlite.org/pragma.html#pragma_synchronous ; Write-Ahead Logging, including the WAL-reset bug, https://www.sqlite.org/wal.html . Consulted 5 October 2026.
10. The Rust Programming Language, Unsafe Rust: https://doc.rust-lang.org/book/ch20-01-unsafe-rust.html . Consulted 5 October 2026.
11. RFC 7946, The GeoJSON Format: https://www.rfc-editor.org/rfc/rfc7946 . Used as format context; this assessment does not certify every possible exported geometry for RFC conformance.

All benchmark records are synthetic. This paper makes no claim of independent endorsement, competitor superiority or exhaustive security verification. Its positive conclusions and limits refer to the fixed version and evidence above.